

Sampling distribution and failure. Given the parameter space $\mathcal{Z}$, we define a uniform sampling distribution:

$$P(Z) = \frac{1}{|\mathcal{Z}|}. \quad (1)$$

Our goal is to estimate the failure probability under P ,

$$\mu := \mathbb{E}_{Z \sim P} [\mathbb{P}(f(Z) = 1)], \quad (2)$$

where for each input Z , $f(Z) \in \{0, 1\}$ is a Bernoulli random variable indicating whether majority voting over K independent stochastic generations yields an incorrect prediction.

Modern LLMs often produce non-deterministic outputs (e.g., under non-zero temperature), especially for reasoning tasks. To obtain more consistent predictions, we adopt self-consistency decoding (Wang et al., 2023): for each input Z , we perform K independent inference runs and aggregate the outputs via majority voting. This reduces output variance by aggregating over diverse reasoning paths, approximating the mode of the model’s output distribution.

Formally, we define the failure indicator as

$$f(Z) := \mathbb{1}\{\text{MajorityVote}(\hat{y}_1(Z), \dots, \hat{y}_K(Z)) \neq y\}, \quad (3)$$

where $\hat{y}_k(Z)$ denotes the model output from the k -th independent inference on input Z , and y is the ground-truth label. Ties are counted as failures.

Datasets and models. Building on GSM-Symbolic (Mirzadeh et al., 2025), we generate approximately 100K instances per template by varying parameter ranges, yielding nine templates in total. Details of all templates used throughout our paper can be found in Appendix G. We notice that for some template-model pairs, failures are extremely rare (i.e., the template is too easy for some model) which is potentially non-zero but undetectable within our feasible evaluation budget. Therefore, we only report results for pairs with a sufficient number of observed failures to support reliable estimation. Details of such excluded pairs are provided in Section C.

Due to compute budget limits, we focus on relatively efficient yet capable LLMs, including Qwen2.5-Math-7B-Instruct (Yang et al., 2024), gpt-oss-20b-low (Agarwal et al., 2025) with low reasoning efforts, and Gemini 2.5 Flash Lite (Comanici et al., 2025). All inferences follow the models’ recommended decoding configurations (e.g., temperature, top- p , and max number of tokens), and details such as system prompts and parsing final answers are provided in Appendix A.

Bootstrapping Inferences. Ideally, evaluating an LLM requires fresh inferences for each generation, since LLM outputs are inherently stochastic (i.e., non-zero temperature), particularly during reasoning. However, this becomes computationally infeasible for evaluating with large sample sizes for rare failures, across multiple models, templates, and random seeds (for coverage). Therefore, we employ *bootstrap resampling* (Efron, 1979) to approximate the stochastic variation of LLM outputs without fresh inference for each input prompt. Specifically, we pre-compute and cache a pool of independent generations for each input, 16 generations for Qwen2.5-Math-7B-Instruct, 40 for gpt-oss-20b-low, and 32 for Gemini 2.5 Flash Lite, which is deemed large enough to approximate the true sampling. We randomly sample K outputs with replacement from this cached pool to simulate independent generations for majority voting. This resampling approximates the distribution of majority-vote outcomes given fresh inferences without additional inference costs.

4 LLM Failures Are Systematic, Not Random

Failures concentrate on a small subset of parameters. We observe that *LLM failures are not random, but exhibit systematic structure*. Within the high-dimensional parameter space $\mathcal{Z}$, a small subset of parameter values disproportionately accounts for a large fraction of

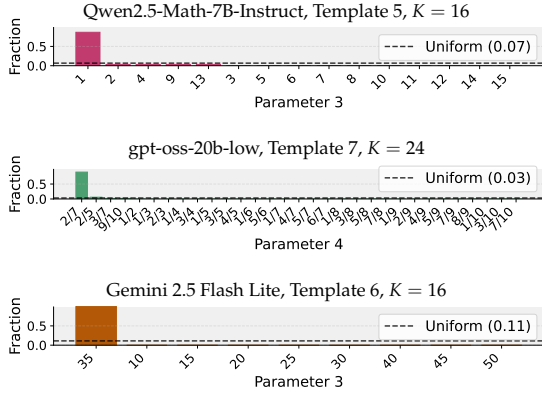


Figure 2: Distribution of failures across parameter values for model–template pairs. Each bar represents the proportion of failures for each parameter value, and the dotted line shows the expected uniform error rate.

Prompt (Template 5)

{Katy} makes {coffee} using teaspoons of sugar and cups of water in the ratio of {15}: {1}. If she used a total of {736} teaspoons of sugar and cups of water, calculate the number of teaspoonfuls of sugar she used.

LLM Output (Qwen2.5-Math-7B-Instruct)

To determine the number of teaspoonfuls of sugar Katy used, we start by

...

Since 1 cup of water is equivalent to 16 teaspoons (assuming standard US measurements

...

Therefore, the number of teaspoonfuls of sugar Katy used is: 360

Answer: 690

Prompt (Template 7)

{Jamie} is juggling at a {circus}. {Jamie} can juggle {168} balls. {1/2} of the balls are {golf} balls, and {2/7} of the {golf} balls are {blue}. How many {blue} {golf} balls are there?

LLM Output (gpt-oss-20b-low)

analysisCompute: total balls 168.
Half are golf balls: 168/2=84
golf balls. Of those, 2/7 are blue:
84*(2/7)=12.assistantfinalJamie has
(12) blue golf balls.
Answer: 24

Figure 3: Example pairs of prompt–generations that illustrate repetitive failures by LLMs. Failure-inducing parameter values and output traces are highlighted in red.

failures. Across models and templates, we observe striking concentration of failures on individual parameter values (Figure 2). We first iterate over all possible inputs for testing models, and aggregate all failures observed. We observe that a single parameter value accounts for the majority of errors: parameter term of water cups = 1 in Template 5 for Qwen2.5-Math-7B-Instruct (86% of failures), parameter fraction of blue ball = 2/7 in Template 7 for gpt-oss-20b-low (Agarwal et al., 2025) (82%), and parameter area of fruits = 35 in Template 6 for Gemini 2.5 Flash Lite (Comanici et al., 2025) (81%). Bar plots for all parameters are given in Appendix Figure 6.

To quantify this concentration, we compute the total variation (TV) distance between the normalized histogram of the failure counts over each parameter dimensions (Z_i such as Z_{Name}) and the uniform distribution (Table 1). The uniform distribution represents a baseline where the failures occur evenly across parameters. Thus, larger distances indicate more pronounced failure concentration, while a value of zero indicates that failures are spread evenly. For each setup, we report the top-3 parameter dimensions with the largest divergence. Across models and templates, we observe consistently large deviations from the uniform distribution, i.e., most Top-1 distances reach 0.9. Furthermore, as we increase

Model	ID	Distance		
		Top1	Top2	Top3
Qwen2.5-Math-7B-Instruct (K = 16)	0	0.900	0.750	0.509
	1	0.914	0.833	0.750
	2	0.538	0.249	0.019
	3	0.472	0.362	0.310
	4	0.339	0.264	0.185
	5	0.940	0.867	0.800
	6	0.879	0.752	0.247
	7	0.798	0.748	0.557
gpt-oss-20b-low (K = 24)	0	0.933	0.850	0.729
	2	0.699	0.541	0.052
	4	0.631	0.508	0.385
	7	0.895	0.633	0.537
	8	0.760	0.599	0.487
Gemini 2.5 Flash Lite (K = 16)	0	0.867	0.850	0.267
	2	0.907	0.750	0.080
	4	0.208	0.133	0.102
	6	0.983	0.889	0.189

Table 1: Total variation distance between failure distributions and uniform baseline. Parameters with the three largest distances are reported for each template.

K , we observe that the TV distance increases (Table 3), which implies that with larger K , the majority of failures becomes more pronounced, with fewer inconsistent failures. Specifically, larger K filters out random errors—failures caused by incidental factors in individual generations—while amplifying systematic failure patterns that persist across multiple samples. These results suggest that LLM failures are not spread randomly across the parameter space, but are instead driven by a small subset of failure-inducing inputs.

Qualitative analysis of repetitive failures. To understand how the failures concentrate in certain regions of the parameter space, we conduct a qualitative analysis of LLM-generated outputs that exhibit repetitive failures, as in Figure 3. We mark in red the input parameter values that frequently induce failures as well as the erroneous reasoning trace. For instance, Qwen2.5-Math-7B-Instruct consistently applies an unjustified unit conversion heuristic when the parameter value term for cup of water = 1 is given in Template 5. gpt-oss-20b-low fails at simple multiplication when 2/7 is given as fraction of blue ball in Template 7. Similarly, Gemini 2.5 Flash Lite fails in 35×43 in Template 6 (Figure 27). These models exhibit such errors repeatedly across multiple generations, as illustrated in Figures 19, 23, and 27. We provide additional qualitative analysis for other template-model pairs in Section I, which show various error types including wrong arithmetic operations, unnecessary approximation, problem misinterpretation, and others. These examples reveal that LLM errors in GSM problems stem from consistent, input-dependent reasoning behaviors, providing insight into the conditions under which models are likely to produce incorrect reasoning trajectories. Despite there being no a priori reason to expect systematic failure patterns to persist across template-model pairs, we find compelling empirical evidence that such patterns arise consistently across most pairs we examine.

5 Learning Failure-Prone Sampling Distributions

Our key finding of the concentration of failures in Section 4 naturally leads to a failure-prone sampling strategy—rather than drawing samples from the full parameter space under P , we concentrate sampling on failure-prone regions. To this end, we leverage the CEM to iteratively refine a proposal distribution Q toward regions of the parameter space where failures are more likely, which we can use to estimate true error probability via importance sampling.

Importance Sampling with Q . To estimate μ defined in Eq. 2, a standard Monte Carlo estimator under P is

$$\hat{\mu}_P = \frac{1}{N} \sum_{i=1}^N f(Z_i), \quad Z_i \sim P.$$

However, when failures are rare, the estimator suffers from high variance relative to μ , requiring large numbers of samples to obtain tight confidence bounds. To improve efficiency, we introduce a proposal distribution Q and estimate μ via importance sampling:

$$\hat{\mu}_Q = \frac{1}{N} \sum_{i=1}^N f(Z_i) \frac{P(Z_i)}{Q(Z_i)}, \quad Z_i \sim Q,$$

which yields an unbiased estimate under the support condition that $Q(Z) > 0$ whenever $\mathbb{P}(f(Z) = 1) P(Z) > 0$. To ensure that importance weights are bounded, we employ defensive sampling (Hesterberg, 1995), which mixes Q with P with mixing coefficient λ .

CEM for learning Q . The variance-minimizing proposal distribution is given by $Q^*(Z) = \frac{\mathbb{P}(f(Z)=1)P(Z)}{\mu}$, which concentrates probability mass on failure regions. The CEM seeks a proposal distribution Q that approximates this optimal distribution by minimizing the Kullback-Leibler divergence, $\text{KL}(Q^* \parallel Q)$. Since Q^* depends on the unknown μ , the CEM iteratively refines Q by sampling from the current proposal and updating toward the empirical failure-weighted distribution proportional to $\mathbb{P}(f(Z) = 1) P(Z)$.